

JavaScript

Varför JavaScript?

Webbläsarens monopol

JavaScript (JS) är det enda programmeringsspråk som alla moderna webbläsare förstår nativt. Om du vill bygga applikationer för webben finns det ingen väg runt det – du *måste* använda JavaScript, eller språk som kompileras till det (som TypeScript).

WebAssembly (Wasm)? Idag kan du köra språk som C# eller Rust direkt i webbläsaren via WebAssembly. Men Wasm kan inte ändra HTML eller prata med DOM-trädet på egen hand. Det krävs fortfarande alltid ett JavaScript-skript som laddar igång det och sköter kontakten med webbsidan. Så oavsett vad – du kommer inte undan JS!

Ett enormt ekosystem

Genom åren har JavaScript vuxit till att bli ett av världens mest populära språk. Via pakethanteraren **npm** har du tillgång till miljontals färdiga kodbibliotek. Det betyder att du sällan behöver uppfinna hjulet på nytt; oavsett om du behöver hantera datum, bygga animationer eller kommunicera med databaser finns det redan stabila verktyg färdiga att använda.

Hög efterfrågan på arbetsmarknaden

Eftersom i princip alla moderna företag har en webbnärvaro eller bygger webbaserade system, är JavaScript-utvecklare ständigt eftertraktade. Att behärska JS öppnar dörren till en enorm arbetsmarknad.

Fullstack

Den största fördelen för en utvecklare idag är att JavaScript tillåter dig att vara en *Fullstack-utvecklare*. Du kan använda exakt samma språk och liknande logik för att bygga det användaren ser i webbläsaren (frontend) som du gör för att bygga logiken och serverhanteringen i bakgrunden (backend).

Webbens tre grundpelare (- jämförelse med en bil):

- **HTML** är chassit och strukturen.
- **CSS** är lacken, formen och inredningen.

- **JavaScript** är motorn, elektroniken och pedalerna som gör att bilen faktiskt går att köra.

Fungerar JavaScript BARA på en webbsida?

Svaret är ett tydligt: Nej.

När JavaScript skapades 1995 av Brendan Eich var det tänkt som ett enkelt skriptspråk för att göra statiska webbsidor lite mer levande (t.ex. skapa enkla animationer eller validera formulär). Men språket har genomgått en enorm evolution. Idag används JavaScript långt utanför webbläsarens gränser.

Node.js – JavaScript på servern

År 2009 tog utvecklare tog loss JavaScript-motorn (V8, samma motor som driver Google Chrome) från webbläsaren och byggde **Node.js**.

- Detta gjorde det möjligt att köra JavaScript-filer direkt på en vanlig dator eller server, precis som man kör Python, C# eller Java.
- Idag byggs enorma, högpresterande Web-API:er och serverapplikationer helt i JavaScript tack vare Node.js.

Mobilapplikationer (iOS & Android)

Med hjälp av moderna ramverk som **React Native** (som bygger på JavaScript) kan utvecklare skriva en enda kodbas i JS och kompilera den till riktiga, nativa appar för både iPhones och Android-telefoner. Appar som Instagram, Skype och Discord använder denna teknik.

Skrivbordsapplikationer (Desktop)

Många av de program du använder på din dator dagligen är faktiskt byggda med webbt teknik och JavaScript under huven via ett verktyg som heter **Electron**.

Applikation	Teknik under huven
Visual Studio Code	JavaScript / TypeScript & Webbläsarmotor
Discord	JavaScript & Webbläsarmotor
Spotify	JavaScript & Webbläsarmotor

Det betyder att den miljö du kodar i varje dag (VS Code) i sig är ett bevis på hur kraftfullt JavaScript är utanför webbsidan.

Vad är JavaScript? (Ett dynamiskt & tolkat språk)

För att förstå hur man skriver effektiv kod måste man först förstå hur språket betar sig under huven. JavaScript har två utmärkande egenskaper: det är *tolkat* och det är *dynamiskt typat*.

1. Ett tolkat språk (Just-In-Time Compilation)

Till skillnad från språk som C# eller C++, där koden måste kompileras om till en färdig .exe-fil eller binärkod innan den kan köras, är JavaScript ett tolkat språk.

- Webbläsaren (eller Node.js) läser koden och kör den **rad för rad, i realtid**.
- Detta kallas JIT-kompilering (*Just-In-Time*).
- **Fördel:** Det går extremt snabbt att ändra i koden, spara och se resultatet direkt utan väntetider.

2. Dynamiskt typat

I ett statiskt typat språk måste du explicit tala om vad en variabel ska innehålla (t.ex. att det är ett heltal eller en textsträng). I JavaScript sköter språket detta automatiskt. Variabler binds till värdetyper under körning, inte i förväg.

```
// Exempel på dynamisk typning:  
let x = 42;           // JavaScript förstår att x är av typen  
'Number'  
x = "Hej klassen!"; // Helt okej! Variabeln x har nu bytt typ till  
'String'
```

Namnförvirringen: Java och JavaScript

En av de vanligaste missuppfattningarna bland nybörjare är att Java och JavaScript är relaterade. **Det är de inte.** De är två helt olika språk med olika användningsområden och filosofier. Namnet "JavaScript" var ett rent marknadsföringsknep på 90-talet eftersom Java var det trendigaste språket just då.

Varning till klassen: Att JavaScript är dynamiskt och förlåtande gör det lätt att lära sig. Men det innebär också att felaktiga beräkningar (t.ex. att råka addera en textsträng med ett nummer) inte upptäcks förrän koden faktiskt körs. Detta är exakt den problematik

som gör att industrin har rört sig mot **TypeScript**, vilket vi kommer att titta på om några dagar!

Datalagring – Variabler & Scope (let vs const)

När vi programmerar behöver vi spara data i minnet. I modern JavaScript har vi lämnat gamla tiders synder bakom oss och använder två specifika nyckelord: `const` och `let`.

`const` (Konstant) – Ditt standardval

`const` används för variabler vars värde **inte** ska skrivas över under programmets gång.

- Detta är ditt absolut viktigaste verktyg för att skriva säker kod.
- Genom att låsa ett värde förhindrar du att andra delar av ditt program råkar ändra på data av misstag.

```
const maxAntalStudenter = 30;  
maxAntalStudenter = 35; // ERROR: Assignment to constant variable.
```

`let` – När förändring krävs

`let` använder du när du vet att värdet *måste* kunna ändras. Det typiska exemplet är en räknare i en loop, eller en statusflagga som slår om från `true` till `false`.

```
let poang = 0;  
poang = poang + 10; // Helt okej!
```

Konceptet Block-Scope

Både `let` och `const` är strikt begränsade till det block de skapas i. Ett block definieras av måsvingar `{ }`. Variabler som skapas inuti ett block är osynliga och skyddade från utsidan.

```
if (true) {  
    const hemlighet = "Det här är dolt";  
}  
console.log(hemlighet); // ERROR: hemlighet is not defined
```

Historisk notis: Undvik var

På internet kommer du se nyckelordet **var**. **Använd det inte.** `var` saknar block-scope och kan "läcka" ut ur if-satser och loopar, vilket historiskt sett har orsakat tusentals svårhittade buggar.

Tre sätt att skriva funktioner

Funktioner är byggstenarna i JavaScript. De samlar kod som gör en specifik uppgift så att vi kan återanvända den. I JavaScript finns det tre olika sätt att deklarera en funktion, och det är viktigt att känna igen alla tre.

1. Function Declaration (Det traditionella sättet)

Detta är det klassiska sättet som påminner mest om andra programmeringsspråk.

```
function beraknaMoms(pris) {  
  return pris * 0.25;  
}
```

- **Egenskap:** Drabbas av *hoisting*. Det betyder att JavaScript internt lyfter upp funktionen till toppen av filen innan koden körs. Du kan tekniskt sett anropa funktionen på rad 2, trots att du inte skrev själva funktionen förrän på rad 10. Detta kan ibland göra kodflödet svårläst.

2. Function Expression (Funktion lagrad i en variabel)

Här skapar vi en anonym funktion och sparar den i en konstant (`const`).

```
const beraknaMoms = function(pris) {  
  return pris * 0.25;  
};
```

- **Egenskap:** Eftersom funktionen ligger i en konstant drabbas den inte av hoisting. Du kan inte använda den förrän efter den rad där den har skapats, vilket tvingar fram ett mer logiskt och linjärt kodflöde.

3. Arrow Functions (Den moderna standarden =>)

Introducerades i och med ES6-uppdateringen (2015) och är idag den absolut mest använda metoden i modern utveckling.

```
const beraknaMoms = (pris) => {  
  return pris * 0.25;  
};
```

- **Egenskap:** En mycket renare och mer kompakt syntax som tar bort mycket av det visuella "bruset" i koden.

Från 4 rader till 1 med Arrow Functions

Arrow Functions är inte bara ett annat sätt att skriva – det har inbyggda funktioner som gör att vi kan skriva extremt elegant och kompakt kod. Det tydligaste exemplet på detta kallas **Implicit Return**.

Det traditionella upplägget (4 rader)

Om vi vill skriva en funktion som tar ett nummer och räknar ut dess kvadrat (numret multiplicerat med sig själv), skriver vi traditionellt så här:

```
function kvadrat(nummer) {  
  const resultat = nummer * nummer;  
  return resultat;  
}
```

Här tvingas vi använda måsvingar `{ }` för att tala om var funktionen börjar och slutar, samt nyckelordet `return` för att skicka tillbaka resultatet.

Den moderna optimeringen (1 rad)

Om en Arrow Function bara ska utföra en enda beräkning och skicka tillbaka resultatet, tillåter JavaScript oss att kasta bort både måsvingarna `{ }` och ordet `return`. JavaScript förstår automatiskt att det som står efter pilen (`=>`) är det som ska levereras tillbaka.

```
const kvadrat = nummer => nummer * nummer;
```

Tre regler för att komprimera Arrow Functions:

1. **Skippa function:** Ersätt med pilen `=>` efter argumenten.
2. **Skippa parenteser vid ett argument:** Om funktionen bara tar ett argument (som nummer ovan) behövs inga parenteser runt det. (Om den tar noll eller flera än ett argument krävs dock parenteser, t.ex. `() => ...` eller `(a, b) => ...`).
3. **Implicit return:** Ta bort `{ }` och `return` om allt får plats på en rad.

Varför är detta branschstandard?

Förutom att det sparar rader, förändrar Arrow Functions inte värdet på nyckelordet `this`. De ärver istället sitt sammanhang. Det gör dem otroligt mycket säkrare att använda när vi skickar runt funktioner som argument, vilket är grundbulten i modern frontend-utveckling och ramverk som **React**.

Moderna Array-metoder

I modern JavaScript använder vi sällan traditionella `for`- eller `while`-loopar för att stega igenom listor med data. Istället använder vi inbyggda array-metoder som tar emot Arrow Functions som argument. Detta gör koden mer deklarativ – vi beskriver *vad* vi vill uppnå istället för exakt *hur* datorn ska loopa.

Här är de tre absolut viktigaste metoderna du måste kunna utantill:

1. `.map()` – Transformera data

`.map()` går igenom en array, förändrar varje element baserat på en funktion, och returnerar en helt ny array med samma längd.

```
const priser = [10, 20, 30];
const priserMedMoms = priser.map(pris => pris * 1.25);
// Resultat: [12.5, 25, 37.5]
```

2. `.filter()` – Sortera ut data

`.filter()` testas varje element i en array mot ett villkor. Om villkoret är sant (`true`) får elementet följa med till den nya arrayen. Om det är falskt sorteras det bort.

```
const aldrar = [15, 22, 17, 45, 12];
const Myndiga = aldrar.filter(alder => alder >= 18);
// Resultat: [22, 45]
```

3. `.reduce()` – Slå ihop data

`.reduce()` används när du vill koka ner en hel array till ett enda slutgiltigt värde (till exempel för att räkna ut summan av alla tal i en lista).

```
const kundvagn = [100, 250, 50];
const totalt = kundvagn.reduce((summa, pris) => summa + pris, 0);
// Resultat: 400
```

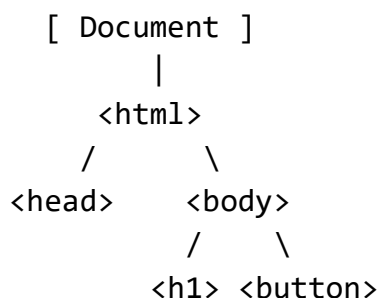
(Nollan i slutet av metoden är startvärdet för beräkningen).

DOM: Att göra webbsidan levande

När JavaScript körs i webbläsaren är dess primära uppgift att interagera med HTML-dokumentet. Detta sker via något som kallas **DOM** (*Document Object Model*).

Vad är DOM?

DOM är en strukturerad, trädliknande representation av din HTML-sida som webbläsaren skapar i minnet. Varje HTML-tagga (<h1>, <p>, <div>) blir till ett "objekt" i det här trädet. JavaScript kan gå in i det här trädet och ändra text, byta färger, ta bort element eller lägga till helt nya komponenter i realtid.



Det interaktiva flödet i 3 steg

Att bygga interaktivitet på en webbsida handlar nästan alltid om att följa denna trestegsraket:

Steg 1: Hämta elementet (Select)

Först måste vi tala om för JavaScript vilket HTML-element vi vill arbeta med. Vi använder den moderna metoden `querySelector`.

```
const knapp = document.querySelector('#min-knapp');
const rubrik = document.querySelector('h1')
```

Steg 2: Lyssna på användaren (Listen)

Vi lägger till en "händelselyssnare" (*Event Listener*) på elementet. Vi berättar för webbläsaren: *"Håll koll på det här elementet, och om användaren gör X, så vill jag att du reagerar"*.

```
knapp.addEventListener('click', () => {  
  // Steg 3 sker här inuti...  
});
```

Steg 3: Agera och förändra (Act)

Inuti vår lyssnare kör vi koden som faktiskt uppdaterar sidan genom att manipulera DOM-trädet.

```
knapp.addEventListener('click', () => {  
  rubrik.textContent = "Du klickade på knappen!";  
  rubrik.style.color = "green";  
});
```